

LOW-LEVEL PROJECT UNDERSTANDING DOCUMENT

AI SDLC Studio

Autonomous Multi-Agent Engine for the Software Development Life Cycle

Version Analyzed: 4 | Document Type: LLD / Project Understanding Document

Prepared for: Engineering Handover / Onboarding

Table of Contents

1. Executive Summary
2. Project Objective & Problem It Solves
3. Technology Stack
4. High-Level System Architecture
5. Project Folder Structure
6. Database Design (All Tables & Fields)
7. Phase 1 — Requirement Agent (Requirement Engineering)
8. Phase 2 — Design Agent (System Architecture / SDD)
9. Phase 3 — Development Agent (Multi-Agent Code Generation)
10. Phase 4 — Testing Agent (Automated Test Generation & Execution)
11. Backend REST API Reference
12. Frontend Application Structure
13. LLM Provider, Prompts & Reliability (Rate-Limit Fallback)
14. Security, Guardrails & Authentication
15. End-to-End Data Flow (Walkthrough of One Project)
16. Setup, Installation & Run Instructions
17. Known Limitations & Notes for New Developers
18. Glossary of Terms

1. Executive Summary

AI SDLC Studio is a full-stack, AI-powered web application that automates the early-to-mid stages of the Software Development Life Cycle (SDLC). A user starts by describing a business problem in plain language, and the system — through a chain of autonomous 'agents' built with LangGraph — progressively turns that idea into a formal Software Requirements Specification (SRS), a System Design Document (SDD) with UML/Mermaid diagrams, working source code for a small full-stack application, and an automated test suite with a test execution report.

The project is organized into four logical phases that mirror a real SDLC: Requirement Engineering, System Design, Development (code generation), and Testing. Each phase is implemented as its own LangGraph 'agent' (a state machine of processing nodes) and each phase ends with a mandatory Human-in-the-Loop approval gate before the project is allowed to move to the next phase. This means the tool is not fully autonomous — a human reviewer must approve or reject the output of each stage, and can add comments that are fed back into the next generation attempt.

The system is built as two independent applications that talk to each other over HTTP: a Python/FastAPI backend that hosts the agents, the database, and the document generators; and a React + TypeScript + Vite frontend that gives the user a chat-style interface to talk to the agents and a set of dashboards to review generated artifacts, diagrams, code, and test reports.

2. Project Objective & Problem It Solves

The tool is designed to solve a common, expensive problem in software teams: translating a vague business idea into a properly structured, traceable set of engineering artifacts (SRS → SDD → Code → Tests) usually takes a Business Analyst, a Solution Architect, and multiple developers working over days or weeks. AI SDLC Studio compresses this into a guided, reviewable, minutes-to-hours pipeline, while still keeping a human 'in the loop' at every gate so nothing ships without sign-off.

2.1 Core Goals

- Convert an unstructured problem statement (typed or uploaded as a document) into an IEEE-830-style SRS.
- Detect gaps/ambiguities in requirements automatically and ask clarifying questions before finalizing.
- Auto-generate a System Design Document: architecture style, 10 Mermaid.js UML diagrams, DB schema, REST API spec, ADRs, and a full requirement-to-design Traceability Matrix.
- Auto-generate real, runnable full-stack source code (FastAPI backend + static frontend) using a coordinated team of specialized 'developer' agents (Backend, Frontend, Database, API-Integration, Documentation, Self-Review).
- Auto-generate and 'execute' a layered test suite (Unit, Integration, API/Contract, Functional, Regression) and produce a downloadable PDF test report.
- Preserve full auditability: every stage stores versioned snapshots, reviewer comments, and activity logs in a relational database.
- Allow every generated artifact (SRS, SDD, code, tests) to be exported/downloaded in multiple formats (PDF, DOCX, Markdown, JSON, ZIP).

3. Technology Stack

3.1 Backend

Layer	Technology	Purpose
Language / Runtime	Python 3.9 – 3.10+	Core backend language
Web Framework	FastAPI + Uvicorn	REST API server
Agent Orchestration	LangGraph	State-machine based multi-step agent pipelines
LLM Integration	LangChain + langchain-google-genai	Talks to the LLM provider
LLM Provider	Google Gemini (gemini-3.6-flash / 1.5 Pro), pluggable OpenAI/Anthropic	Generates requirements, design, code, tests
ORM / Database Access	SQLAlchemy 2.0	Data persistence layer
Database	SQLite (dev default), PostgreSQL-ready (via DATABASE_URL)	Stores projects, versions, logs, tests
Schema Migrations	Alembic	Versioned DB migrations
Document Export	ReportLab (PDF), python-docx (Word)	Generates downloadable SRS/SDD/Test reports
Agent Checkpointing	langgraph-checkpoint-sqlite	Persists LangGraph state so multi-turn chat & approvals can resume

3.2 Frontend

Layer	Technology	Purpose
Framework	React 18 + TypeScript	SPA UI
Build Tool	Vite 5	Dev server & bundler
Styling	Tailwind CSS	Utility-first styling
Icons	lucide-react	Iconography
Diagram Rendering	Mermaid.js 10	Renders UML/architecture diagrams generated by the Design Agent
HTTP Layer	Native Fetch API (custom wrapper in services/api.ts)	Talks to backend, injects X-API-Key header

4. High-Level System Architecture

The system follows a classic client–server architecture, but the 'server' side is itself composed of four chained AI agents, each implemented as an independent LangGraph state machine with its own checkpointing (so a paused, awaiting-human-approval conversation can be resumed later).

4.1 Component View

- React SPA (frontend/) — single dashboard with 4 tabs: Dashboard, Requirements (chat), Design (diagrams/spec), Development (code + tests).
- FastAPI backend (backend/app/main.py) — exposes ~45 REST endpoints, guarded by a static X-API-Key header, orchestrating the 4 agents via service modules.
- Requirement Agent (agents/requirement_agent.py) — LangGraph pipeline that runs the chat, extracts a ProjectDocument, detects gaps, validates quality, and finalizes a backlog.
- Design Agent (agents/design_agent.py) — LangGraph pipeline that reads the approved requirements and produces a full System Design Document (SDD) with diagrams, DB design, API design and ADRs.
- Development Agent / Orchestrator (agents/development/orchestrator.py) — LangGraph pipeline that plans tasks, fans work out to specialized coding sub-agents, merges/validates output, generates tests, executes them, and packages a downloadable ZIP.
- Relational Database (SQLite/Postgres) — stores Projects, Requirement/Design/Development versions, Human Reviews, Activity Logs, Test Cases & Reports, and backlog items (Epics/Features/User Stories/Acceptance Criteria).
- LLM Provider Layer (services/llm_provider.py) — a swappable factory that returns a LangChain chat model (Gemini by default) and silently falls back to a deterministic MockChatModel if no API key is configured or the real provider fails — this keeps the whole SDLC pipeline runnable/demoable even with zero API quota.

4.2 System Context Diagram (textual)

```

User (Architect / Business Analyst)
|  problem statement, file upload, approvals
v
React Frontend (Vite, :3000)
|  REST/JSON + X-API-Key header
v
FastAPI Backend (:8000)
|
|-- Requirement Agent          (LangGraph -> SRS)
|-- Design Agent               (LangGraph -> SDD + Mermaid diagrams)
|-- Development Orchestrator   (LangGraph -> 6 sub-agents, tests, zip)
|-- Document Generator         (ReportLab / python-docx -> PDF/DOCX/MD)
|
v
Google Gemini LLM API (pluggable: OpenAI / Anthropic / Mock)
|
v
SQLite / PostgreSQL Database (business data + LangGraph checkpoints)

```

5. Project Folder Structure

The repository root contains two independent projects — backend/ and frontend/ — plus a top-level README.md.

5.1 Backend (backend/app/)

main.py — FastAPI app entry point; declares every REST route (~45 endpoints).

database.py — SQLAlchemy engine/session setup + the LangGraph SQLite checkpointer.

models.py — SQLAlchemy ORM models — all 21 database tables.

schemas.py — Pydantic request/response schemas (SRS, ProjectDocument, review payloads, etc.).

agents/requirement_agent.py — Phase 1 LangGraph pipeline (Requirement Agent).

agents/design_agent.py — Phase 2 LangGraph pipeline (Design Agent).

agents/development_agent.py — Legacy/simple development agent helper module.

agents/state.py, design_state.py, development_state.py — TypedDict state schemas for each LangGraph pipeline.

agents/development/orchestrator.py — Phase 3 & 4 LangGraph pipeline (multi-agent Development + Testing orchestrator).

agents/development/agents/*.py — Six specialized coding sub-agents: backend, frontend, database, api_integration, documentation, self_review.

agents/development/core/*.py — Shared engines: prompt_manager, template_engine, code_snippet_store, merge_validation, artifact_generator, memory_manager, rules_engine.

agents/development/planning_engine.py, context_builder.py — Task planning and context-assembly helpers used before code generation starts.

prompts/requirement_prompts.py, design_prompts.py — Centralized system prompts used by the Requirement & Design agents.

services/project_service.py — CRUD + completeness-scoring logic for projects/requirements.

services/agent_service.py — Bridges REST layer to the Requirement Agent LangGraph graph (chat + approvals).

services/design_service.py — Bridges REST layer to the Design Agent LangGraph graph.

services/development_service.py — Bridges REST layer to the Development Orchestrator graph; task/log/artifact/test retrieval.

services/llm_provider.py — LLM factory + MockChatModel fallback + get_content_text helper.

services/document_generator.py — Builds PDF/DOCX/Markdown exports for SRS, SDD and Test Reports.

services/document_parser.py — Extracts plain text from uploaded requirement documents (pdf/docx/txt).

services/input_guardrails.py — Prompt-injection regex filter + LLM-based relevance classifier.

services/json_repair.py — Repairs malformed JSON returned by the LLM before schema validation.

services/requirement_validator.py — Rule-based completeness/quality scoring engine for the SRS (Stage-3 gate).

alembic/ — Database migration scripts (6 revisions).

5.2 Frontend (frontend/src/)

App.tsx — The entire application shell (~3,600 lines) — tabs for Dashboard / Requirements / Design / Development, chat UI, diagram viewers, document viewers, approval controls.

components/DevelopmentPanel.tsx — Dedicated panel (~1,000 lines) rendering the multi-agent development pipeline: task list, per-agent status, execution logs, generated files, test results.

services/api.ts — Typed fetch wrapper (api object) — every backend call + TypeScript interfaces (Project, SRSOutput, TestCase, etc.).

main.tsx / index.css / vite-env.d.ts — App bootstrap, Tailwind base styles, Vite type declarations.

6. Database Design (All Tables)

All persistence is done through SQLAlchemy models in backend/app/models.py, versioned with Alembic. There are 21 tables grouped into 5 logical clusters. SQLite is the default (sdlc_studio.db); DATABASE_URL in .env can point to PostgreSQL instead. A second SQLite file (sdlc_checkpoints.db) is used exclusively by LangGraph to persist agent conversation/graph state (separate from business data).

6.1 Core Project & Review Tables

Table	Key Columns	Purpose
projects	id, name, description, current_phase, status	Root entity. current_phase = REQUIREMENT/DESIGN/DEVELOPMENT. status = IN_PROGRESS/AWAITING_APPROVAL/APPROVED/REJECTED.
human_reviews	project_id, phase, stage, status, comments, reviewer_name	Audit trail of every approve/reject decision at every gate.
activity_logs	project_id, action, details, timestamp	Generic project-level activity/audit feed shown on the Dashboard.
version_snapshots	project_id, version_num, document, reviewer_comments	Historical snapshots of the ProjectDocument at each requirement version.
project_agent_states	project_id (PK), current_state, document, messages, memory, missing_info, validation_attempts	1:1 live snapshot of the Requirement Agent's in-flight state (chat memory, extracted doc, gaps) for quick reads by the /status endpoint.

6.2 Requirement (SRS) Tables

Table	Key Columns	Purpose
requirements	project_id, approval_status	One row per project representing the 'requirement document' container.
requirement_versions	requirement_id, version_num, raw_srs (JSON), reviewer_comments	Every generated/approved SRS snapshot — enables full version history (v1.0, v2.0, v2.1...).
epics / features / user_stories / acceptance_criteria	epic_id/feature_id/story_id/ac_id, title, description, linked_requirement_ids, source_type, confidence	The finalized Agile backlog synthesized in Stage 4 of the Requirement Agent, each item carrying provenance ('source') back to the original requirement text.

6.3 Design (SDD) Tables

Table	Key Columns	Purpose
design_documents	project_id, requirement_version_id, approval_status	Container linking a design document to the exact requirement version it was generated from.

Table	Key Columns	Purpose
design_versions	design_document_id, version_num, raw_sdd (JSON), reviewer_comments	Every generated SDD snapshot (architecture, diagrams, DB design, API design, ADRs, traceability matrix).

6.4 Development (Code Generation) Tables

Table	Key Columns	Purpose
development_versions	project_id, design_version_id, version_num, raw_manifest (JSON), artifact_zip_path, artifact_paths, parent_version_id, changed_files	One row per code-generation run; supports incremental/targeted revisions via parent_version_id + changed_files.
development_reviews	version_id, status, comments, rejected_modules, quality_score, security_score	Human approval/rejection decisions on generated code, optionally scoped to specific rejected modules.
development_logs	project_id, action, details, timestamp	High-level activity feed specific to the development phase.
development_execution_logs	project_id, version_id, agent_name, stage, node_name, started_at, completed_at, duration, prompt_tokens, completion_tokens, retry_count, status, error_message	Fine-grained, per-LangGraph-node execution trace used to render the 'Execution Logs' tab (like a build log).
development_tasks	task_id, module, description, owner_agent, dependencies, priority, status, requirement_id, design_section_id	The task list produced by the Task Distribution Engine — each task traces back to a REQ-ID and a design section.
development_agent_outputs	agent_name, output_payload (JSON), status	Raw file output captured from each of the 6 coding sub-agents before merge.
development_artifacts	artifact_type, file_path, checksum	Pointers to on-disk deliverables: source_zip, database_scripts, api_docs, project_report, change_log.

6.5 Testing Tables

Table	Key Columns	Purpose
test_cases	test_case_id, module, test_type, test_scenario, preconditions, test_steps, expected_result, actual_result, status, source_snippet	Individual generated test case, including the generated pytest source code itself. test_type ∈ UNIT/INTEGRATION/API/FUNCTIONAL/REGRESSION.
test_execution_reports	total_unit/passed_unit/failed_unit ... (per category), overall_status, report_pdf_path	Aggregated pass/fail counters per test category for a development version, plus the path to the generated PDF test report.

7. Phase 1 — Requirement Agent

File: backend/app/agents/requirement_agent.py (942 lines). Implemented as a LangGraph StateGraph named 'workflow', compiled with a SQLite checkpointer (compiled_graph = workflow.compile(checkpointer=sqlite_checkpointer)). The checkpointer is what allows the graph to 'pause' at a human-approval node and resume later from the exact same point once the human submits a decision through the REST API.

7.1 Purpose

Turns a free-text problem statement (typed in chat or extracted from an uploaded document) into a canonical, structured ProjectDocument (business goals, personas/actors, workflows, functional/non-functional/business-rule requirements, assumptions, constraints, dependencies, risks) and then into a full Agile backlog (Epics → Features → User Stories → Acceptance Criteria), all while gating progress behind 4 human-approval checkpoints.

7.2 The 4-Stage Gated Pipeline

Stage	LangGraph Nodes Involved	What Happens	Gate
1. Extraction	input_validation → conversation_state_manager → prompt_generator_and_llm → extraction_node	Chat turns are folded into a RequirementMemory object; once enough signal exists, an LLM call (EXTRACTION_SYSTEM_PROMPT) produces the canonical ProjectDocument JSON (goals, actors, workflows, requirements, assumptions, constraints, dependencies, risks).	awaiting_extraction_approval — human reviews the extracted document.
2. Gap Detection	gap_detector_node	A second LLM call (GAP_DETECTOR_SYSTEM_PROMPT) inspects the requirement list for missing NFRs, ambiguity, or contradictions and returns a structured list of GAP-### items, each with a suggested clarifying question and a 'blocking' flag.	awaiting_gap_approval — human can approve gaps as-is or route back into waiting_for_clarification for more chat.
3. Quality Validation	validation_node (uses services/requirement_validator.py)	A deterministic, rule-based scoring engine (no hardcoded thresholds beyond a pass mark) checks completeness across requirement types, business context, and workflow coverage, returning a numeric Quality Score. Per the README, the project targets a minimum score of 70 to proceed.	awaiting_validation_approval.
4. Backlog Finalization	finalizer_node	A third LLM call (FINALIZER_SYSTEM_PROMPT) synthesizes Epics, Features, User Stories, and Given/When/Then Acceptance Criteria from the approved requirements, ensuring every functional requirement is linked to at least one User Story (100% coverage) and every backlog item carries a 'source' provenance object.	awaiting_finalization_approval — final sign-off; project then transitions current_phase → DESIGN.

7.3 Supporting Mechanisms

- RequirementMemory / ProjectDocument (schemas.py) — Pydantic models that define the exact shape of everything the agent must collect and produce; used both to validate LLM output and to drive the frontend forms.
- Logical Completeness Engine — `calculate_logical_requirement_completeness()` in `project_service.py` scores elicitation depth without hardcoded keyword counts; feeds the 'Requirement Completeness %' shown on the Dashboard.
- `waiting_for_clarification` node — loops back into `conversation_state_manager`, i.e. if gaps are marked blocking, the chat re-opens so the user can answer the `suggested_question` before re-attempting extraction.
- Input Guardrails (`services/input_guardrails.py`) — every chat message / uploaded document first passes a regex-based prompt-injection detector and an LLM-based 'is this software-development related' relevance classifier before being handed to the agent.
- `json_repair.py` — LLM responses are not always perfectly valid JSON; this module attempts to fix common issues (trailing commas, unescaped quotes, truncation) before Pydantic validation.
- Document Upload — `/requirements/upload` accepts PDF/DOCX/TXT; `document_parser.py` extracts raw text which is then fed into the same chat pipeline as if the user had typed it.
- Fast-Track — a project can be approved and immediately handed off to the Design Agent from the Requirements screen via the 'Approve & Move to Design Agent →' action, without manually visiting a separate screen.

7.4 Routing Logic

Each node pairs with a `route_after_*` function (e.g. `route_after_extraction`, `route_after_gap_approval`) that inspects the current state and returns the name of the next node — this is what makes the pipeline a graph rather than a straight line: on REJECTED review status the graph typically routes back to an earlier generation node (e.g. back to `extraction_node`) carrying the reviewer's comments as extra context for the next LLM call, instead of dead-ending.

8. Phase 2 — Design Agent

File: backend/app/agents/design_agent.py (938 lines). Also a LangGraph StateGraph, compiled as compiled_design_graph with the same SQLite checkpointer pattern. It is triggered by POST /api/projects/{id}/design/generate once the Requirement phase is APPROVED.

8.1 Purpose

Reads the approved requirements/ProjectDocument and produces a complete System Design Document (SDD): recommended architecture style with rationale, technology stack selection, module breakdown, database design, REST API design, security design, deployment topology, 10 Mermaid.js UML diagrams, Architecture Decision Records (ADRs), and a Requirement Traceability Matrix mapping REQ-ID → Module → API Route → DB Entity → ADR.

8.2 Linear Generation Chain (9 nodes, no gate between them)

- input_validation — verifies an approved ProjectDocument/requirements exist for this project.
- requirement_analyzer — re-reads the requirements to build the design-time context.
- architecture_planner — chooses an architecture style (e.g. Modular Monolith with Domain Separation, or Event-Driven Microservices) and writes the technical rationale.
- technology_selector — picks concrete technologies (e.g. PostgreSQL 16, Redis 7, JWT auth) consistent with the chosen architecture.
- module_designer — breaks the system into modules/components and their responsibilities.
- database_designer — designs table schemas (users, user_sessions, orders, audit_logs, etc.), columns, types, nullability, PKs/FKs, unique constraints.
- api_designer — designs REST endpoints: methods, request/response payloads, error codes, JWT Bearer auth flow.
- security_designer — authentication/authorization flow, security policies, logging & exception-handling strategy.
- deployment_planner — deployment/infrastructure topology across Dev, Staging, and Multi-AZ Production.
- diagram_generator — produces the 10 Mermaid.js diagram strings from all of the above.
- sdd_generator — assembles everything into the final SDD JSON object matching the SoftwareDesignDocument schema.

8.3 The 10 Mermaid.js Diagram Views

#	Diagram	SDD Field
1	C4 System Context	system_context_diagram_mermaid
2	Use Case / System Scope	use_case_diagram_mermaid
3	Container & Component Architecture	component_diagram_mermaid
4	Class Diagram	class_diagram_mermaid
5	Runtime Sequence (per user story)	sequence_diagram_mermaid
6	Activity / Process Flow	activity_diagram_mermaid
7	Entity-Relationship (ER)	er_diagram_mermaid
8	Deployment / Infrastructure Topology	deployment_diagram_mermaid
9	General Flow Diagram	flow_diagram_mermaid
10	DB Relationship Cardinality Map	db_relationship_diagram_mermaid

8.4 Gated Sub-Pipeline (mirrors the Requirement Agent pattern)

- `sdd_generator` → `awaiting_design_generation_approval` → `post_generation_handler` — human reviews the freshly generated SDD draft.
- `design_gap_detector_node` → `awaiting_design_gap_approval` → `post_gap_handler` — detects design-level gaps (e.g. missing NFR coverage in the architecture).
- `design_validation_node` → `awaiting_design_validation_approval` → `post_validation_handler` — validates structural completeness/consistency of the SDD (this powers the `design_completeness_percent` and `architecture_quality_score` metrics surfaced on the Dashboard).
- `design_finalization_node` → `awaiting_design_finalization_approval` → `post_finalization_handler` — final sign-off; SDD v1.0 is persisted and the project becomes eligible to start the Development phase.

8.5 Resilience: `build_fallback_sdd()`

If a project reaches the DESIGN phase but, for any reason, a full SDD was never generated/persisted (e.g. an LLM outage), `main.py`'s `/status` endpoint calls `agents/design_agent.py`'s `build_fallback_sdd()` to synthesize a minimal-but-valid SDD directly from the approved `ProjectDocument`, so the frontend never has to handle a completely empty design screen.

9. Phase 3 — Development Agent (Multi-Agent Code Generation)

File: backend/app/agents/development/orchestrator.py (1,526 lines — the largest module in the codebase). This is the most sophisticated pipeline: a LangGraph StateGraph that coordinates six specialized 'developer' sub-agents, each of which independently calls the LLM to write real source files, followed by automated merge, review, documentation, packaging, test generation, and test execution — all before a human ever sees the code.

9.1 Orchestrator Pipeline (in execution order)

Node	Responsibility
input_validation	Confirms an APPROVED SDD exists for the project before allowing development to start.
development_context_builder	Bundles the approved SRS + SDD into a compact 'development_context' object passed to every sub-agent (see context_builder.py).
project_planner	Uses planning_engine.py to draft an overall implementation plan.
task_distribution_engine	Breaks the plan into individual DevelopmentTask rows (module, owner_agent, dependencies, priority, requirement_id, design_section_id) and fans them out to the 6 coding sub-agents (see 9.2).
build_validator	Runs merge_validation.py — checks that files returned by each sub-agent are syntactically sane and don't conflict, before merging into a single 'merged_files' map.
run_self_review	self_review_agent_node — produces advisory quality/security/architecture-compliance scores and a findings list (e.g. CORS policy present, parameterized DB connections).
final_documentation_refinement	documentation_agent_node — generates README.md and user guides referencing the actual generated file structure.
artifact_generator	core/artifact_generator.py — zips the merged source tree and writes auxiliary artifacts: database_scripts, api_docs, project_report, change_log.
test_case_generator	Synthesizes layered pytest test cases (see Section 10) from the manifest, backend routes, and requirements.
test_executor	'Executes' (validates/records outcomes for) the generated test cases and writes a TestExecutionReport with per-category pass/fail counts.
human_approval	Pauses the graph (interrupt) for a human to Approve / Reject / Request-Revision, optionally scoped to specific rejected_modules.
post_approval_handler	On approval: persists the final DevelopmentVersion + marks project DEVELOPMENT/APPROVED. On rejection: routes back into the sub-agent nodes, but ONLY re-runs the modules listed in rejected_modules (a 'targeted revision', not a full regeneration) — see 9.3.

9.2 The Six Specialized Coding Sub-Agents

Sub-Agent	File	Generates
Backend Agent	agents/backend_agent.py	app/main.py — FastAPI entry point, using the fastapi_app template + a JWT-auth code snippet from code_snippet_store.py.
Frontend Agent	agents/frontend_agent.py	frontend/index.html — the generated application's UI entry layout.

Sub-Agent	File	Generates
Database Agent	agents/database_agent.py	app/database.py — DB connection/schema code, using the sqlite_db template.
API Integration Agent	agents/api_integration_agent.py	No new files — instead it regex-parses backend routes (@app.get/post/put/delete) out of the Backend Agent's output and cross-checks them against the Frontend and Database outputs for contract consistency (e.g. every frontend fetch call has a matching backend route).
Documentation Agent	agents/documentation_agent.py	README.md and usage docs, built from the readme_md template and the real generated directory structure.
Self-Review Agent	agents/self_review_agent.py	No files — produces quality_score, security_score, requirement_coverage, architecture_compliance and a findings[] list used in the DevelopmentReview record.

Each coding sub-agent is prompt-driven via core/prompt_manager.py (versioned prompts, e.g. 'BackendAgent v2', 'FrontendAgent v1') and core/template_engine.py (boilerplate skeletons such as fastapi_app, html_entry, sqlite_db, readme_md) so the LLM only has to fill in the project-specific logic rather than reinvent boilerplate every run.

9.3 Targeted / Incremental Revision

A key design decision: when a human rejects a development version, they can reject specific modules only (rejected_modules, e.g. ["backend"]). Every sub-agent node checks ``if rejected and "<its module>" not in rejected: return {}`` — i.e. it skips regeneration and keeps its previous output untouched. This is tracked in the database via `DevelopmentVersion.parent_version_id` and `changed_files`, giving a lightweight diff/versioning model without needing full git integration.

9.4 Observability

Every single node execution is timed and recorded into `development_execution_logs` (agent_name, stage, node_name, duration, prompt/completion tokens, retry_count, status, error_message) via `record_node_execution()`. The frontend's `DevelopmentPanel.tsx` renders this as a live 'build log' so a user can see exactly which agent is running, how long it took, and whether it succeeded — similar to a CI/CD pipeline view.

10. Phase 4 — Testing Agent (Automated Test Generation & Execution)

Testing is not a separate LangGraph graph — it is the tail end of the Development Orchestrator (test_case_generator and test_executor nodes), but it is substantial enough, and has its own dedicated database tables and migration (d481f9a01e23_add_testing_agent_tables.py), to be treated as its own SDLC phase in this document.

10.1 Test Categories Generated

Category	Basis for Generation
Unit	One test scaffold per generated module (e.g. test_<module>_initialization) verifying the module loads/initializes correctly.
Integration / API (Contract)	One test per detected backend route (test_route_<path>_contract), validating the route contract identified by the API Integration Agent.
Functional	System-level workflow tests (test_system_integration_flow, test_user_functional_workflow) tied back to user stories/requirements.
Regression	test_regression_safety — a safety-net test re-run on every subsequent revision to catch regressions introduced by targeted module revisions.
Security	Counted separately in the TestExecutionReport (total_security/passed_security/failed_security) alongside the self-review agent's security_score.

10.2 Test Data Model

- TestCase — stores test_case_id, module, test_type, test_scenario, preconditions, test_steps, test_input, expected_result, actual_result, status (PASS/FAIL/SKIPPED), duration, and the actual generated pytest source_snippet.
- TestExecutionReport — one row per DevelopmentVersion aggregating total/passed/failed counts across all 6 categories (unit, integration, api, functional, regression, security) plus an overall_status (PASSED/FAILED/PARTIAL) and a path to a generated PDF report.

10.3 API Surface

- GET /development/tests — list test cases, optionally filtered by ?type=UNIT|INTEGRATION|API|FUNCTIONAL|REGRESSION.
- GET /development/tests/report — the aggregated TestExecutionReport for the latest version.
- GET /development/tests/download/pdf — a formatted PDF test report (via development_service.generate_test_report_pdf_bytes, built on document_generator.py / ReportLab).

Frontend: DevelopmentPanel.tsx renders a 'Testing' tab showing per-category pass/fail summaries, individual test cases with their generated source code, and a 'Download Test Report (PDF)' button.

11. Backend REST API Reference

All endpoints are served under FastAPI, documented live at /docs (Swagger UI). Every request except /, /health, /docs, /openapi.json, /redoc requires a header X-API-Key: <STUDIO_API_KEY> (or ?api_key= query param for file-download links).

11.1 Projects & Requirements

Method	Endpoint	Description
POST	/api/projects	Create a new project.
GET	/api/projects	List all projects.
GET	/api/projects/{id}	Get one project.
POST	/api/projects/{id}/chat	Send a chat message / problem statement to the Requirement Agent.
POST	/api/projects/{id}/requirements/upload	Upload a document (PDF/DOCX/TXT) as requirement input, after guardrail checks.
GET	/api/projects/{id}/requirements/state	Raw Requirement Agent LangGraph state (memory, gaps, active stage).
POST	/api/projects/{id}/requirements/approve	Approve current requirement stage / fast-track to Design.
POST	/api/projects/{id}/requirements/reject	Reject current stage with reviewer comments.
GET	/api/projects/{id}/requirements/download/{pdf docx markdown json}	Export the SRS.
GET	/api/projects/{id}/status	Aggregated project status: agent states, SRS, SDD, computed metrics, activity logs.
GET	/api/projects/{id}/history	Version history of the requirement document.

11.2 Design Agent

Method	Endpoint	Description
POST	/api/projects/{id}/design/generate	Trigger SDD generation.
GET	/api/projects/{id}/design/state	Raw Design Agent LangGraph state.
GET	/api/projects/{id}/design	Latest design document + version.
POST	/api/projects/{id}/design/approve	Approve design stage / finalize SDD v1.0.
POST	/api/projects/{id}/design/reject	Reject design stage with comments.
GET	/api/projects/{id}/design/versions	Design version history.
GET	/api/projects/{id}/design/download/{pdf docx markdown json}	Export the SDD.

11.3 Development & Testing

Method	Endpoint	Description
POST	/api/projects/{id}/development/start	Trigger multi-agent code generation.
POST	/api/projects/{id}/development/approve	Approve generated code (may target specific modules).
POST	/api/projects/{id}/development/reject	Reject generated code.
POST	/api/projects/{id}/development/revise	Request a targeted revision (equivalent to reject with rejected_modules).
GET	/api/projects/{id}/development	Development state + version history.
GET	/api/projects/{id}/development/tasks	Task list produced by the Task Distribution Engine.
GET	/api/projects/{id}/development/agents	Per sub-agent status.
GET	/api/projects/{id}/development/logs	Fine-grained execution logs (per LangGraph node).
GET	/api/projects/{id}/development/files	Generated file tree/content.
GET	/api/projects/{id}/development/artifacts	List of compiled artifacts (zip, db scripts, docs, report).
GET	/api/projects/{id}/development/artifacts/download?type=	Download one artifact by type.
GET	/api/projects/{id}/development/download	Download the full generated-code ZIP package.
GET	/api/projects/{id}/development/tests	List generated test cases (optional ?type=filter).
GET	/api/projects/{id}/development/tests/report	Aggregated test execution report.
GET	/api/projects/{id}/development/tests/download/pdf	Download the test report as PDF.

11.4 Utility

Method	Endpoint	Description
GET	/	API status / welcome JSON (no auth required).
GET	/health	Health check (no auth required).
GET	/docs	Swagger UI (no auth required).

12. Frontend Application Structure

The frontend is a single-page React application (frontend/src/App.tsx, ~3,600 lines) with one large stateful root component (App()). There is no router library — navigation is done purely with local useState tabs, which keeps the app simple but means the whole UI lives in one file plus one large sub-component (DevelopmentPanel.tsx).

12.1 Top-Level Navigation

- Dashboard — project overview: current phase/status, computed metrics (completeness %, coverage %, quality score, API/table/diagram counts), activity log, and a 3-way 'Deliverables' viewer (SRS / SDD / Code) with per-format export buttons.
- Requirements — the chat interface for Phase 1: message thread, file-upload control, per-stage approval UI (Extraction/Gap/Validation/Finalization), and an SRS section-by-section reader (11 category tabs).
- Design — the Phase 2 workspace: architecture summary, the 10-tab Mermaid diagram viewer (rendered via the MermaidChart component using the mermaid npm package), database design tables, API design tables, ADR cards, and the traceability matrix, plus approve/reject controls.
- Development — delegated to <DevelopmentPanel />: task list, per-sub-agent status cards, live execution logs, generated file browser, test case browser with pass/fail badges, and approve/reject/revise (with module targeting) controls.

12.2 Key Frontend Building Blocks

- MermaidChart component — wraps the mermaid.js library to render any *_diagram_mermaid string returned by the backend into an SVG diagram, with a unique id per chart instance to avoid re-render collisions.
- services/api.ts — a single typed `api` object exposing one method per backend route (listProjects, createProject, getProjectStatus, sendChatMessage, reviewRequirements, generateDesign, reviewDesign, uploadDocument, getTests, getTestReport, ...), all going through apiFetch() which injects the X-API-Key header automatically.
- Locking logic — isDesignTabUnlocked() / isDevelopmentTabUnlocked()-style guards prevent a user from jumping to the Design or Development tab before the prior phase is APPROVED, mirroring the backend's phase gating.
- Polling — the Dashboard periodically calls GET /status to refresh phase/metrics while an agent is running, so long-running LLM calls (extraction, SDD generation, multi-agent code generation) are reflected in near-real time without WebSockets.

13. LLM Provider, Prompts & Reliability

13.1 Provider Selection (services/llm_provider.py)

`get_llm()` reads `DEFAULT_LLM_PROVIDER` (default 'gemini') and available API keys (`GEMINI_API_KEY` / `OPENAI_API_KEY` / `ANTHROPIC_API_KEY`) from environment variables and returns a LangChain `BaseChatModel`. Priority order: configured provider → any other provider with a valid key → `MockChatModel`. Setting `MOCK_MODE=true` forces the mock model unconditionally (useful for offline demos/CI).

13.2 MockChatModel — the 'Zero Downtime' Fallback

This is one of the more distinctive engineering decisions in the codebase. Rather than letting the whole pipeline fail when Google GenAI's free tier returns HTTP 429 (rate limited) or no API key is configured, `llm_provider.py` defines a `MockChatModel` (a LangChain-compatible chat model implemented locally) that pattern-matches the incoming system prompt (e.g. detects 'PlanningEngine'/'ProjectPlanner', 'compile...srs', 'update the structured requirement memory') and returns a deterministic, schema-valid JSON stub appropriate to that stage. This keeps every downstream Pydantic validation, LangGraph routing, and document export path fully exercised and demoable even with zero live LLM quota.

13.3 Prompt Organization

- `requirement_prompts.py` / `design_prompts.py` — centralized, reusable system prompts (e.g. `MEMORY_UPDATER_SYSTEM_PROMPT`, `CHAT_AGENT_SYSTEM_PROMPT`) shared across nodes.
- Inline prompts — `EXTRACTION_SYSTEM_PROMPT`, `GAP_DETECTOR_SYSTEM_PROMPT`, `FINALIZER_SYSTEM_PROMPT` are defined directly at the top of `requirement_agent.py` for the 3 major LLM calls in that pipeline.
- `core/prompt_manager.py` — a versioned prompt registry for the Development sub-agents (e.g. `prompt_manager.get_prompt("BackendAgent", "v2")`), allowing prompts to evolve without touching the sub-agent code.
- Every generation prompt explicitly instructs the model to return raw JSON with no markdown fences and no preamble, and `json_repair.py` provides a second line of defense against near-valid JSON.

14. Security, Guardrails & Authentication

14.1 API Authentication

A single static API key (STUDIO_API_KEY env var, default 'default_secret_key_12345' for local dev) is required on every request via the X-API-Key header (or ?api_key= query param, used for direct-download links that a browser opens in a new tab). This is enforced globally by a FastAPI dependency (verify_api_key) attached at the app level, with an allow-list for /, /health, and the docs routes. This is a simple shared-secret scheme suitable for a single-tenant/local-dev tool — it is not per-user authentication and has no session/JWT layer of its own (JWT is only something the Design/Development agents generate as part of the output application's design, not something this Studio app itself uses to authenticate its own users).

14.2 Input Guardrails (services/input_guardrails.py)

- Prompt-Injection Filter — a regex bank matches phrases like 'ignore previous instructions', 'jailbreak', 'DAN mode', 'developer mode enable', etc., and rejects the request with HTTP 400 before it ever reaches the LLM.
- Relevance Classifier — a lightweight LLM call classifies whether the input is actually about software/SDLC topics; irrelevant input is rejected with HTTP 400, preventing the tool from being used as a general-purpose chatbot.

14.3 CORS

CORS is restricted to a small allow-list (http://localhost:5173, http://127.0.0.1:5173, http://localhost:3000) — i.e. only the local Vite dev server origins — with allow_credentials=True and all methods/headers permitted from those origins.

15. End-to-End Data Flow — Walkthrough of One Project

1. User clicks 'New Project' in the frontend → POST /api/projects creates a Project row (current_phase=REQUIREMENT, status=IN_PROGRESS).
2. User types a problem statement in the Requirements chat → POST /api/projects/{id}/chat → agent_service.run_chat_step() invokes the Requirement Agent's LangGraph graph, which updates RequirementMemory turn by turn.
3. Once enough information is gathered, the graph auto-advances to extraction_node, producing a ProjectDocument, and pauses at awaiting_extraction_approval — the frontend shows an Approve/Reject panel.
4. User approves → gap_detector_node runs → pauses at awaiting_gap_approval (with clarifying questions if any gap is blocking) → user approves/answers → validation_node scores the document → pauses at awaiting_validation_approval → user approves → finalizer_node builds the Epics/Features/Stories/ACs backlog → pauses at awaiting_finalization_approval → user gives final sign-off.
5. On final approval, project_service marks the Requirement phase APPROVED, current_phase → DESIGN, and (via the fast-track button or automatically) design_service.start_design_generation() kicks off the Design Agent graph.
6. The Design Agent runs its 9-node linear chain (architecture → tech stack → modules → DB → API → security → deployment → diagrams → SDD assembly), then goes through its own 3 additional gates (generation/gap/validation/finalization approvals) exactly like the Requirement Agent.
7. On Design approval, current_phase → DEVELOPMENT and the user (or an automatic trigger) calls POST /development/start, invoking the Development Orchestrator.
8. The orchestrator plans tasks, fans them to the 6 coding sub-agents in parallel/sequence, merges + validates + self-reviews + documents the code, zips artifacts, generates layered tests, 'executes' them, and pauses at human_approval with a full DevelopmentVersion, execution log trail, and TestExecutionReport ready for review.
9. User reviews generated code/tests in the Development tab; on APPROVE, post_approval_handler finalizes the version and the project is complete for this iteration. On REJECT (optionally scoped to specific modules), the graph loops back and re-runs only the rejected sub-agents, producing an incremental DevelopmentVersion linked via parent_version_id.
10. At any point the user can export SRS/SDD as PDF/DOCX/Markdown/JSON, and download the generated code as a ZIP or the test report as a PDF — all served as file-download responses straight from document_generator.py / development_service.py.

16. Setup, Installation & Run Instructions

16.1 Prerequisites

- Python 3.10+
- Node.js 18+
- Git 2.30+

16.2 Backend

```
cd backend
python -m venv .venv    (then activate it)
pip install -r requirements.txt
Create backend/.env with GOOGLE_API_KEY / GEMINI_API_KEY, STUDIO_API_KEY,
DATABASE_URL=sqlite:///./sdlc_studio.db
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
Backend runs at http://127.0.0.1:8000 ; Swagger docs at /docs
```

16.3 Frontend

```
cd frontend
npm install
Create frontend/.env with VITE_API_BASE=http://localhost:8000/api and VITE_STUDIO_API_KEY=<same key
as backend>
npm run dev
Frontend runs at http://localhost:3000 (Vite default may show 5173 unless configured; both origins
are already CORS-allowed)
```

17. Known Limitations & Notes for New Developers

- Single shared API key, not per-user auth — fine for a personal/demo tool, would need real auth (OAuth/JWT sessions) before multi-tenant production use.
- MockChatModel fallback means the app can appear to 'work' with placeholder/deterministic output when no valid LLM key is present — always check backend console logs for '[LLM Provider] ... Falling back to MockChatModel' when debugging unexpected generic output.
- App.tsx is a single ~3,600-line file with no component-level code splitting or router — straightforward to run, but any new frontend contributor should budget time to read the file's useState/useEffect blocks carefully before editing, since state is tab-scoped rather than route-scoped.
- Development Agent sub-agent 'time.sleep(1.0)' calls are simulated pacing (to produce readable, staggered execution-log timestamps for the UI) rather than real work — they are harmless but worth knowing about when profiling performance.
- Test 'execution' in test_executor is largely about generating and recording structured pytest-style cases and a report, not necessarily spinning up a live pytest runner against the generated app inside a sandbox — treat generated tests as a scaffold/starting point to run for real, not a guarantee that the generated app has been dynamically verified.
- SQLite is the default datastore for both business data (sdlc_studio.db) and LangGraph checkpoints (sdlc_checkpoints.db) — fine for local/dev; DATABASE_URL supports swapping the business DB to PostgreSQL, but the LangGraph checkpointer is wired specifically to SQLite in database.py.
- Because every phase persists full JSON snapshots per version (requirement_versions.raw_srs, design_versions.raw_sdd, development_versions.raw_manifest), the database will grow with every regeneration — worth adding cleanup/archival policy for long-running projects.

18. Glossary of Terms

Term	Meaning
SRS	Software Requirements Specification — the Phase 1 deliverable.
SDD	System/Software Design Document — the Phase 2 deliverable, includes all UML diagrams.
LangGraph	A library for building stateful, multi-step LLM applications as graphs of nodes with conditional routing and pausable/resumable execution via checkpoints.
Checkpoint	The mechanism LangGraph uses to persist graph state to SQLite so a paused (interrupt) graph can resume exactly where it left off after a human decision.
Human-in-the-Loop Gate	A node that calls interrupt() to pause the graph and wait for a human APPROVE/REJECT decision submitted via a REST endpoint.
ProjectDocument	The canonical Pydantic schema representing all elicited requirement data for a project.
Traceability Matrix	A table mapping each functional requirement to the module/API route/DB entity/ADR that implements it, proving full coverage.
ADR	Architecture Decision Record — a structured card capturing Context, Decision, Alternatives, and Trade-offs for one architectural choice.
Manifest	The list of files (path, module, owner_agent, purpose, dependencies) that the Development Orchestrator plans to generate for a project.
Targeted Revision	Re-running only the sub-agents responsible for modules a human explicitly rejected, instead of regenerating the whole codebase.
Mock Mode	MOCK_MODE=true env flag that forces all LLM calls to return deterministic stub JSON instead of calling a real LLM provider.

— End of Document —